

type-pm-mech2 入門

制約生成, `let` の完全性, `matcher` まで

type-pm-mech2 project

2026 年 8 月 20 日版

基準実装: `main` (本文作成開始時の commit `7fe112f`)

この文書について

この文書は、Lean 4 リポジトリ `type-pm-mech2` の現行実装を、型推論や定理証明に初めて触れる読者にも追えるように説明する。中心となる問いは次の三つである。

1. 式を調べると得られる、仮の結果型 `target`、必ず満たす等式 `hard`、後で分類する使用可能性の要求 `pending` とは何か。
2. 一つの定義を複数の型で使える多相 `let` で右辺を先に閉じると、なぜ「型付けが存在すれば推論が必ず成功する」という性質（完全性）の証明が難しくなるのか。
3. 現行実装はその問題をどう解き、どこまでを証明済みとしているのか。

後半では、Paper 1 用の検証例（fixture）として実装されている、値の分解方法を記述する再帰的 `multiset matcher`、とくに `multisetWithListArgument = lambda list. multiset` という補助式の意味と、この式が「制約からまだ決まらない型の形を先回りして推測しない」方針（no-guess 方針）の下で、単独では推論に失敗する理由も説明する。

旧リポジトリと隣接する論文について

この文書は `type-pm-mech2` の Lean コードを正本とする。兄弟リポジトリ `type-pm-mech` は旧実装であり、旧 `SourceTyping`, `inferRaw`, `wBridgeCheck`, `terminal audit`（推論後の再検査）を現行体系へ持ち込まない。また、`../type-pm-paper` の TeX には旧実装を説明する記述が残っているため、現行実装の API 一覧としては使わない。現状確認には本書、設計と進捗を統合したリポジトリ直下の `README.md`、および Lean コードを用いる。ここで API とは、ほかのファイルから利用するために公開された定義や定理の入口をいう。

目次

この文書について	i
第 1 章 まず全体像をつかむ	1
1.1 何を機械化しているのか	1
1.2 先に覚える基本語	1
1.3 M0 から M5 まで	2
1.4 三つの層を混同しない	4
第 2 章 制約生成：target, hard, pending	5
2.1 Generated という三つ組	5
2.2 関数適用を一つ生成してみる	5
2.3 なぜ pending をすぐ等式にしないのか	6
2.4 小さな例： $((\lambda z. x) 0)$	7
第 3 章 多相 let は何をしているのか	8
3.1 value とは let の右辺のこと	8
3.2 full-cut 方式	8
3.3 PrincipalBlockClosure	8
3.4 一般化と scheme	9
3.5 具体例を最後まで追う	9
3.6 interface equations	10
第 4 章 なぜ let の完全性証明が難しかったのか	11
4.1 実行版は一つを選び、関係版は任意の代表を許す	11
4.2 単純な大域的 rename では直らない	11
4.3 必要なのは文字列一致ではなく意味の一致	12
第 5 章 M2 の最終解法：四つの層	13
5.1 第 1 層：entailed alignment	13
5.2 第 2 層：body を同じ context へ運ぶ有限 rename	13
5.3 第 3 層：visible closure graph	14
5.4 第 4 層：well-formed supply と freshness	14
5.5 whole let の組み立て	14

第 6 章	定理が公開推論へつながるまで	16
6.1	証明の主な鎖	16
6.2	公開 API	16
6.3	負例にも完全性が効く	17
第 7 章	M3 から M5 までの現在地	18
7.1	M3：宣言と通常と呼出し	18
7.2	M4：pattern と matcher	18
7.3	M5：評価と型安全性	19
7.4	一般 pattern function と MNode	19
第 8 章	multisetWithListArgument を理解する	21
8.1	Paper 1 の multiset 定義との対応	21
8.2	三つの式の役割	21
8.3	なぜ lambda 版単独は現在拒否されるのか	22
8.4	設計上の選択肢	22
8.5	別問題：kernel で大きな推論を計算すること	23
第 9 章	コードを読むための案内	24
9.1	最初に読むファイル	24
9.2	検証コマンド	25
第 10 章	まとめと再開地点	26
10.1	M2 について覚えておく四文	26
10.2	次の作業は M2 ではなく M4/M5	26
付録 A	現行実装の型規則一覧	27
A.1	この一覧の読み方	27
A.2	生成 block を組み立てる共通操作	27
A.3	M2–M3 の式の規則	28
A.4	M4 の user pattern 規則	29
A.5	M4 で追加された式の規則	30
A.6	最終的な Typing 規則	33
付録 B	用語集	34
付録 C	旧説明から現行説明への対応	36
版の基準		37

第 1 章

まず全体像をつかむ

1.1 何を機械化しているのか

type-pm-mech2 は、Type-PM という型体系を Lean 4 で機械化するプロジェクトである。ここで機械化とは、型や推論規則をプログラムとして定義するだけでなく、「推論成功なら規則上も型が付く」ことや「規則上型が付くなら推論が成功する」ことまで、Lean の定理として検査することをいう。前者を健全性、後者を完全性と呼び、後で改めて定義する。

通常の間数型言語では、型推論、つまりプログラムを調べてその型を自動的に求める処理は、式に `Int` や $\alpha \rightarrow \beta$ のような型を付ける。 α と β はまだ分からない型を表す型変数であり、矢印は「 α 型の値を受け取り β 型の値を返す関数」を表す。Type-PM ではさらに `matcher` を扱う。 `matcher` は、値をどのように分解・観察できるかを記述する実行時の値である。そのため型にも次の二つの見方が現れる。

`matcher producer` 実際に作られた `matcher` 値の型。本書では概略 `Matcher  $\kappa \tau$` と書く。

`matcher slot` `matcher` を使う場所が要求する型。本書では概略 `Slot  $\kappa \tau$` と書く。

τ は対象値の型であり、 κ は capability である。capability は、`matcher` が入力値のどの形を観察できるかを表す型情報である。 `producer` から `slot` への変換は単なる型等式ではないため、普通の単一化だけでなく checking という別の処理が必要になる。これが後で `pending` を分離する理由である。

1.2 先に覚える基本語

この先で繰り返し使う語を、コード上の英語名より先に日本語で説明する。

source 式	型を調べる対象となる入力言語の式である。本書で単に「式」と書くときも、通常はこの式を指す。
context	式のその場所から参照できる変数と、各変数に与えられた型を並べた環境である。たとえば無名関数 (lambda) の本体では、lambda が束縛した引数の型が context へ追加される。自由変数とは、その式自身の lambda や let では導入されず、外側の context から参照する変数である。
代入	型変数を具体的な型で一斉に置き換える対応である。たとえば $[\alpha \mapsto \text{Int}]$ は、 α を <code>Int</code> で置き換える。

単一化	複数の型等式を同時に成り立たせる代入を計算する処理である。
関数適用	関数を引数へ使う式である。たとえば $f\ x$ は関数 f を引数 x へ適用する。
制約 block	一つの式についてまとめて解く制約の一まとまりである。現行実装では、生の結果型 <code>target</code> 、必ず満たす等式 <code>hard</code> 、後で分類する使用可能性の要求 <code>pending</code> を持つ。三つの詳しい意味は次章で説明する。
型 scheme	型変数を利用時ごとに選べる形で束ね、一つの定義を複数の型で使えるようにした多相型である。この束ねる操作を量化といい、コード名は <code>Scheme</code> である。
signature	<code>nil</code> や <code>cons</code> のように <code>data</code> 値を作る名前 (data constructor) や、 <code>add</code> のような組込み演算 (primitive) と、それらの型 <code>scheme</code> を登録した有限の表である。
公開推論	利用者が式の型を求めるために呼ぶ、リポジトリの正式な推論関数 <code>infer</code> である。
supply	次に作る新しい型変数と <code>capability</code> 変数へ何番を与えるかを記録する二つのカウンタである。
well-formed な supply	<code>context</code> にすでに現れるすべての自由変数より後の番号から、新しい変数名を割り当てる <code>supply</code> である。これにより、生成した新しい名前が <code>context</code> 中の古い名前と衝突しない。
closure	一つの制約 block を解いて得た代入と結果型をまとめた記録である。実行時に関数本体と環境を組にした「関数 closure」とは別物である。
回帰検証	具体例と期待結果を固定し、将来の変更で以前の正しい動作が壊れていないかを確かめる検査である。

証明済みの性質を表す四つの語も、次の意味で用いる。

健全性	公開推論が成功したなら、推論器とは独立に定義した型付け規則でも型が付くこと。
完全性	推論器とは独立な型付けが存在するなら、公開推論が必ず成功すること。
主要性	公開推論が返す型が最も一般的で、ほかの型付け結果をその型への代入で得られること。
受理同値	二つの制約 block について、一方を解けることと他方を解けることが同値であること。

1.3 M0 から M5 まで

実装は段階に分けられている。2026 年 8 月 20 日時点の要点は表 1.1 のとおりである。

表 1.1: 実装段階の要約

段階	主題	状態	意味
M0	型, 代入, 局所 checking	完了	matcher を含む基礎的な型合成と checking を検証済み.
M1	lambda, application, 制約 block	完了	独立した型付け, 停止する単一化, 推論の健全性・完全性・主要性を検証済み.
M2	型 scheme, 多相 let	完了	一般の入れ子 letE を含む M2-M3 の source 式で完全性・受理同値・主要性を検証済み.
M3	data, primitive, signature	一部	Bool/List, 値を作る constructor, 組込み演算 primitive, ifE, 有限 signature を実装済み. M4 と結ぶ掲載例全体の回帰が残る.
M4	pattern, matcher, fix	一部	最終構文と統合推論に加え, pattern function 本体の照合を外側から隔離する実行時 node である MNode と Paper 2 の pair 回帰を実装済み. source 宣言と runtime 本体表の双方向対応も pair で証明した. 各 runtime 本体は保存 scheme の標準的な一つの具体化について検査証拠を持つが, 全具体化に対する主要性までは未証明である. 生成変数の由来を追う証明は, 公開 M4 elaboration まで完了した. 大きい検証例の正確な公開推論と M4 全域の完全性・主要性が残る.
M5	評価と型安全性	一部	順序付き探索, 型付き環境, 通常・再帰 closure, 任意の型付き関数位置を持つ application, map まで実装済み. ここで型安全性とは, 型の付いた実行が型の不整合で停止しない性質である. source/runtime 定義表の整合性証明を必須にする MNode 全式評価器と, Paper 2 の全式回帰も実装済みだが, 評価器全体の関係的意味論と型安全性は未証明である. source 多相 let から実行時型付けへの橋と user-defined matcher の型安全性も残る.

「M2 が完了」の正確な意味

M2 では、独立な型付けがあれば公開推論が必ず成功すること（完全性）と、公開推論の型がほぼかのすべての型付け結果の元になること（主要性）を証明済みである。ただし推論を始めるカウンタ（開始 supply）は、その場所で参照できる変数の環境（context）にすでに現れるすべての自由変数より後から、新しい番号を割り当てなければならない。この安全な開始条件を「context に対して well-formed である」と呼ぶ。公開推論はこの条件を満たす `context.initialSupply` から必ず始める。条件を破る任意の開始 supply は未解決の宿題ではなく、API の仕様が受け付ける範囲の外である。

1.4 三つの層を混同しない

現行実装には、役割の異なる三つの層がある。

1. **制約生成**：source 式から **Generated** を作る。
2. **制約の閉包**：生成した制約を解き、主要な結果を得る。
3. **独立した型付け**：実行可能推論器を定義に含めず、型付け規則だけで作る命題 **Typing** で、source program が型を持つことを述べる。

健全性・完全性・主要性は、前節で定義した意味で、この三つの層を結ぶ性質である。

第 2 章

制約生成：target, hard, pending

2.1 Generated という三つ組

Lean では一つの式から得られる情報を次の構造にまとめる.

```
structure Generated where
  target    : Ty
  hard      : List Equation
  pending   : List CheckObligation
```

それぞれの意味は次のとおりである.

target (生の結果型)

target は, 式自身を構文に沿って調べた直後の結果型である. まだ代入を適用していないため, 型変数を含んでよい. 「最終的に確定した型」ではなく, 制約 block が返そうとしている型の入口である.

hard (無条件に必要な等式)

hard は, checking 変換の種類に関係なく必ず成り立つ型等式の列である. たとえば関数適用 $f\ x$ では, f の型が「引数型から結果型への関数」でなければならない. この外形は hard 等式に入る.

pending (後で判定する checking 要求)

pending は, ある生の型が期待型として使えるかを後で判定する要求の列である. 本書では $s \Leftarrow e$ と書き, 「source 型 s を expected 型 e として使えるか」と読む. 普通の型等式になる場合もあれば, matcher producer から slot への特殊変換になる場合もある.

2.2 関数適用を一つ生成してみる

関数式 f から

$$\langle t_f, H_f, P_f \rangle$$

が、引数式 x から

$$\langle t_x, H_x, P_x \rangle$$

が得られたとする。新しい型変数 d, r を割り当てると、 $f\ x$ の生成結果は概略

$$\begin{aligned} target &= r, \\ hard &= H_f ++ H_x ++ [t_f = d \rightarrow r], \\ pending &= P_f ++ P_x ++ [t_x \Leftarrow d] \end{aligned}$$

となる。ここで $++$ は列の連結である。

関数の外形 $t_f = d \rightarrow r$ は常に必要なので hard である。一方、引数 t_x を d として使う方法は、単なる一致か matcher 変換かがまだ分からないので pending へ置く。

「 r checks against d 」の読み方

$[r \Leftarrow d]$ は「式の生の型 r を、使用箇所が要求する型 d として使えるか後で調べる」という記録である。 $r = d$ と最初から決める記号ではない。同様に $[p \Leftarrow d]$ は、source 型が p である別の式に同じ期待型 d が課されたことを表す。

2.3 なぜ pending をすぐ等式にしないのか

matcher producer と slot の間には、capability を調べる特殊変換がある。したがって「source 型を expected 型として使える」という要求は、常に型等式になるとは限らない。説明のため、次の三つの名前と型が context に登録済みだと仮定し、末尾の二つの関数適用を比べる。最初の三行は説明用の型的前提であり、source プログラム本体は最後の二行である。

```
inc : Int -> Int
useMatcher : Slot Any Int -> Bool
m : Matcher Any Int

inc 0
useMatcher m
```

ここで Any は「特定の分解能力を要求しない」capability である。m は実際に作られた matcher なので producer 型 `Matcher Any Int` を持ち、useMatcher は matcher を受け取る場所なので slot 型 `Slot Any Int` を引数に要求する。

どちらの適用でも、制約生成時には引数位置の型として新しい変数 d を置き、引数について「引数の生の型を d として使えるか」という pending を作る。違いが分かるのは hard を解いた後である。

1. `inc 0` では、関数側の hard

$$\text{Int} \rightarrow \text{Int} = d \rightarrow r$$

を解くと $d = \text{Int}$ になる。したがって pending は $\text{Int} \Leftarrow \text{Int}$ となり、普通の等式 $\text{Int} = \text{Int}$ として処理できる。

2. `useMatcher m` では、関数側の `hard`

$$(\text{Slot Any Int}) \rightarrow \text{Bool} = d \rightarrow r$$

を解くと $d = \text{Slot Any Int}$ になる。pending は

$$\text{Matcher Any Int} \Leftarrow \text{Slot Any Int}$$

となる。これは producer 型と slot 型の外側が異なるため型等式ではないが、matcher が調べる対象値の型 `Int` と capability の `Any` が適合するので、matcher から slot への特殊変換として受理できる。

もし生成直後にすべての pending を等式へ変えると、二番目は

$$\text{Matcher Any Int} = \text{Slot Any Int}$$

を要求してしまう。Matcher と Slot は型の最外形を作る異なる記号（型 constructor）なので、これは解けず、本来受理すべき `useMatcher m` を誤って拒否する。反対に、すべてを matcher 変換として扱えば、一番目の普通の `Int` 引数を処理できない。

生成時点では d がまだ新しい型変数であり、expected 型の外側が `Int` なのか `Slot` なのか確定していない。実際の関数式はさらに別の関数適用や `let` を含むこともあり、その場合は `hard` をまとめて解くまで外側の形が分からない。そこで現行設計は、まず `hard` を最も一般的に解き、その結果を pending へ適用してから、普通の等式か matcher 変換かを分類する。

pending は制約を無視する仕組みではなく、**どの checking 規則を使うかだけを延期する仕組み**である。分類後に得られる型と capability の等式は、最終的にすべて解かなければならない。

この「`hard` を解く前に型の外側を推測しない」方針を本書では *no-guess* と呼ぶ。

これはあらゆる意味的な解を探索する方式ではない。`hard` の主要解で checking 分岐を固定した後、に受理できるものを *branch-stable acceptance* と呼ぶ。branch-stable とは、後から型を具体化しても checking の分岐選択をやり直さない、という意味である。

2.4 小さな例：((lambda z. x) 0)

外側の変数 x の型を p とする。式

$$((\lambda z. x) 0)$$

を生成する。 z の型を a 、適用が期待する引数型を d 、結果型を r とすると、

$$\begin{aligned} \text{target} &= r, \\ \text{hard} &= [a \rightarrow p = d \rightarrow r], \\ \text{pending} &= [\text{Int} \Leftarrow d] \end{aligned}$$

となる。`hard` を解くと $a = d$ かつ $p = r$ である。その後 pending を調べると $d = \text{Int}$ になる。したがって式全体の結果は p 、つまり x と同じ型である。

ここで r は「value の型」そのものを最初から意味していたのではない。 r は右辺式の `target` として新しく割り当てた仮の結果変数であり、制約を解いた後に p と同じだと分かる。

第 3 章

多相 let は何をしているのか

3.1 value とは let の右辺のこと

Lean の `Expr.letE value body` における `value` は、実行時の値一般を指す言葉ではない。次の source 構文で等号の右側にある式 e_1 を指すフィールド名である。

$$\text{let } y = e_1 \text{ in } e_2$$

このとき `value` は e_1 、`body` は e_2 である。

3.2 full-cut 方式

現行実装の `let` は full-cut 方式である。cut は「境界で切る」という意味であり、右辺の制約 block を `body` より先に完全に閉じる。手順は次のとおりである。

1. 外側 context で右辺 `value` を生成する。
2. 右辺の `hard` を解き、`pending` を分類・解決して、ほかの解の元になる最も一般的な closure（主要 closure）を得る。
3. closure の代入を外側 context へ適用する。
4. closure の結果型を、更新後 context に自由でない変数について量化し、型 scheme を作る。この量化処理を一般化と呼ぶ。
5. その scheme を context 先頭へ追加して `body` を生成する。
6. 右辺の内部制約は親 block へ出さず、外側 context へ見せる必要がある影響だけを有限個の等式（interface equations）として返す。

この実行可能な手順は `Source/Elaboration.lean` の `elaborate` に現れる。一方、独立した関係 `Elaborates.letE` は実行可能推論器を呼ばず、任意の主要 closure とその吸収性の証明を受け取る。

3.3 PrincipalBlockClosure

`PrincipalBlockClosure` は、最も一般的な制約 block の解を表す記録である。ここで「最も一般的」とは、ほかの解をこの解への追加の代入で得られることをいう。この記録は二段階の解をまとめる。

1. pending から hard へ安全に移せる要求がなくなるまで処理する. この反復を飽和と呼び, 飽和後の hard を解く最も一般的な代入を記録する.
2. まだ pending に残った要求を分類して得る等式 (残余等式) を解く, 最も一般的な代入を記録する.

二つの代入を順に適用できる一つの代入へまとめたものが closure substitution であり, それを target へ適用した型が closure target である. 吸収的であるとは, 後から同じ制約を解く代入を合成しても, 選んだ主要代表が正規化された形で保たれる性質である.

「最も一般的」であっても, 変数名の代表が文字どおり一意になるわけではない. 等式 $p = r$ は r を p へ消しても, p を r へ消しても解ける. この小さな自由度が, let 完全性証明の中心問題になる.

3.4 一般化と scheme

一般化は, 結果型に現れる変数のうち, 更新後 context に自由に現れないものだけを量化する処理である. その結果である型 scheme は, 一つの型を複数の具体的な型として再利用できる多相型である. scheme 内の量化変数は名前ではなく位置で表す. これを bound-index scheme, つまり「量化変数を番号で指す scheme」と呼ぶ. たとえば恒等関数の型 $\alpha \rightarrow \alpha$ を空 context で一般化すれば $\forall \alpha. \alpha \rightarrow \alpha$ になる.

一方, 右辺の結果型が外側変数 x の型 p なら, p は context にも現れるため量化しない. let $y = \dots$ で得た y はその外側 x と同じ単相型 p を持つ.

3.5 具体例を最後まで追う

次の式を考える.

```
lambda x.
  let y = ((lambda z. x) 0) in
    ((lambda w. 0) y)
```

前章の計算から, let 右辺の結果型は外側 x の型 p である. 右辺を閉じて p は外側 context に自由に現れるため, y の scheme は量化されず, 単相 p になる. body の $(\text{lambda } w. 0)$ は $q \rightarrow \text{Int}$ という型を持ち, $y : p$ へ適用することで $q = p$ となる. body の結果型は Int である. したがって lambda 全体の型は

$$p \rightarrow \text{Int}$$

である. top-level で p を一般化して読むなら

$$\forall p. p \rightarrow \text{Int}$$

である.

$\forall a.\text{Int}$ ではなく $\forall a.a \rightarrow \text{Int}$

全体は `lambda x. ...` なので、引数 x を受け取る関数である。body が常に 0 を返しても、`lambda` の矢印は消えない。したがって主要型は $\forall a.a \rightarrow \text{Int}$ であり、 $\forall a.\text{Int}$ ではない。後者は実質 `Int` と同じで、関数でない。

3.6 interface equations

右辺を閉じた代入 S が外側 context の自由変数を変更した場合、その影響を body より外へ伝える必要がある。ただし右辺の全 `hard` と `pending` を再び外へ出すと `full-cut` ではなくなる。そこで、context の各自由型変数 x について

$$x = S(x)$$

だけを作る。capability 変数についても同様である。これが `Context.interfaceEquations` である。

親の後続代入 T がこの interface を解くことは、context に見える各変数について

$$T(x) = T(S(x))$$

が成り立つことと同値である。interface は右辺の制約全部でも、body へ渡す checking 要求でもない。外側から観測できる有限な効果だけである。

最後に `Generated.fromLet effects body` は

$$\begin{aligned} target &= body.target, \\ hard &= effects ++ body.hard, \\ pending &= body.pending \end{aligned}$$

を返す。右辺 value の `pending` は境界を越えないが、body 自身が作った `pending` は外側 block に残る。

第 4 章

なぜ let の完全性証明が難しかったのか

4.1 実行版は一つを選び、関係版は任意の代表を許す

公開推論器は実行可能な `unify` を使い、右辺に対して一つの closure を計算する。しかし関係 `Elaborates.letE` は、条件を満たす任意の主要かつ吸収的な closure を許す。完全性を示すには、関係側がどの正しい代表を選んでも、公開推論器が選ぶ代表へ運べることを示さなければならない。

先ほどの右辺では `hard` から $p = r$ が得られた。次の二つはどちらも正しい主要な向きである。

$$S_L = [r \mapsto p], \quad S_R = [p \mapsto r].$$

S_L では閉じた context は $x : p$ 、右辺結果も p である。 S_R では閉じた context は $x : r$ 、右辺結果も r である。意味は同じだが、body の生成に使う変数名が違う。interface も一方では $p = p$ という自明式、他方では $p = r$ という非自明な別名等式になり得る。別名等式とは、異なる変数名が同じ型を表すことを記録する等式であり、以下では `alias equation` と呼ぶ。

4.2 単純な大域的 rename では直らない

最初に考えたいのは「一方の変数名を全体で置き換えればよい」という案である。本書では、型変数と `capability` 変数の名前を一对一に付け替える操作を `rename` と呼ぶ。しかし全単射な変数名変更は、自明式 $x = x$ を別の自明式 $y = y$ へ移すことしかできない。非自明式 $x = y$ へは変えられない。現行リポジトリにはこの失敗を `GlobalRenamingCounterexample` として Lean で保存している。

ほかにも次の強すぎる案が反例で退けられた。

- 常に左側だけへ別名等式を足す。実際には代表選択により足す側が変わる。
- `hard` だけを共通化し、`pending` は文字どおり同じだと要求する。body の `pending` にも代表名の差が残る。
- body だけを先に `rename` し、任意の外側 frame で同値だとする。外側制約が変数の同一性を観測できる。

ここで `frame` とは、生成済み制約 block を穴へ入れる周囲の制約構造である。完全性の帰納法で再利用するには、裸の block だけでなく適切な周囲へ入れても受理を保つ必要がある。

4.3 必要なのは文字列一致ではなく意味の一致

二つの pending が別の変数名を含んでも、周囲の hard を解くすべての代入の下で同じ要求になればよい。これが `EntailedObligationEq H left right` である。 *entailed* は「hard 理論 H から必ず従う」という意味である。

たとえば hard に $p = r$ があれば

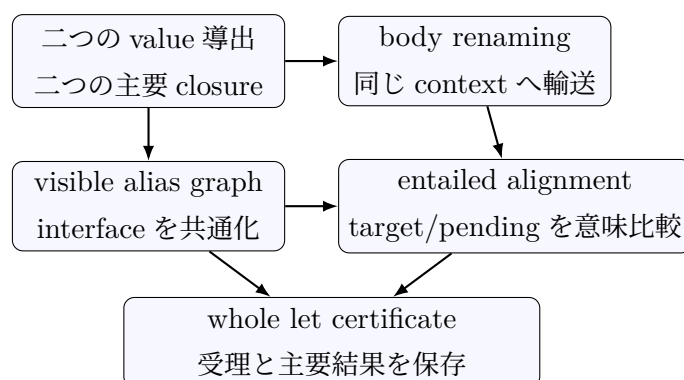
$$p \Leftarrow d \quad \text{と} \quad r \Leftarrow d$$

は文字どおり違うが、 H を解く任意の代入の下では同じになる。この関係は、checking 分岐の分類、hard への昇格、飽和、残余等式、最終受理まで輸送できる。

第 5 章

M2 の最終解法：四つの層

最終証明は、一つの強い rename 定理で全部を済ませない。役割の違う四層を組み合わせる。以下で support とは、型・制約・代入などに実際に現れる有限個の変数名の集合をいう。freshness とは、新しく割り当てた変数名が既存の support と衝突しない性質をいう。



5.1 第 1 層：entailed alignment

EntailedGeneratedAlignment は、二つの Generated について次を要求する。

1. hard の解集合が同じである。
2. その hard を解く任意の代入の下で target が同じになる。
3. 対応する pending が同じ要求になる。

さらに SupportedEntailedAlignmentCertificate は、終了 supply, 左右に追加する有限な別名等式、それらが既存名と衝突しないことを示す freshness, frame の support をまとめる。lambda, application, tuple, constructor, primitive, conditional は、この証明書を構文に沿って合成できる。

5.2 第 2 層：body を同じ context へ運ぶ有限 rename

二つの closure の結果 context は $x:p$ と $x:r$ のように文字どおり異なり得る。body の帰納仮定を使うには、context を同じにしなければならない。そこで closure の有限 support から変数対応 ρ を作る。

ρ には二つの重要な性質がある.

`bodyContext_exact` 左の closed context と一般化 scheme を ρ で移すと、右のものと文字どおり一致する.

`FixesAtOrAbove` value 終了後に新しく割り当てる将来の変数名を ρ が変更しない.

後者により、左 body の導出を同じ source AST、同じ数値 supply のまま右 context へ輸送できる。これは source の de Bruijn index を変える操作ではない。型変数と capability 変数の有限な名前変更である。

5.3 第 3 層：visible closure graph

body を運ぶ rename だけでは、外側 `fromLet` の interface 差を消せない。そこで二 closure の変数対応のうち、closed context から実際に観測でき、かつ名前が動く辺だけを有限個の「左の変数名と右の変数名の対応」として取り出す。この有限な対応表を visible closure graph、つまり外側 context から見える closure 変数対応と呼ぶ。

必要な側へ fresh な別名等式を追加すると、左右の interface は共通の hard 理論

$$I_L ++ I_R$$

を表すようになる。別名等式は左右非対称でよい。一つの全体 swap を要求しない。

先ほどの例で、一方が $p = p$ 、他方が $p = r$ なら、欠けている側へだけ fresh な別名を追加して共通理論へ移す。ただし context や body で既に使われている端点を fresh だと呼んではならない。そのため graph は全 closure support ではなく、context から見える移動辺に限定する。

5.4 第 4 層：well-formed supply と freshness

freshness は「この変数名はたまたま違う」という主張ではない。source 生成変数の由来と数値区間を使って証明する。開始 supply が context に対して well-formed なら、source が新しく作る変数は半開区間 $[start, finish)$ にあり、古い context 変数と衝突しない。

closure の局所性により、block support にない変数は closure substitution が変更しない。これらを合わせると次が言える。

- graph alias の fresh 端点は value の割当区間内にある。
- 対応する側の interface と body context にはその端点が現れない。
- value 区間の alias と、後続 body が作る alias は衝突しない。
- body 用 rename は body 以後の fresh 名を固定する。

5.5 whole let の組み立て

body の証明書は rename 後の座標で作られている。そのため左 body 側の alias 列を ρ の逆向きにたどり、元の座標へ pull back する。pull back とは、移した名前を元の名前空間へ戻す操作である。

最後に、左右の graph alias, pull back した body alias, body の entailed alignment をまとめて、元の二つの `Generated.fromLet` 全体に対する `SupportedEntailedAlignmentCertificate` を構成する。

この手順が重要なのは、反例で偽になった「孤立した body rename は任意の周囲で無害」という中間命題を仮定しない点である。value 導出, closure, transport 済み body 導出を保持したまま, interface と body を一緒に組み立てる。

第 6 章

定理が公開推論へつながるまで

6.1 証明の主な鎖

現行の無条件な M2 証明入口は `Source/FullM2Completion.lean` に集約されている。大きな流れは次のとおりである。

1. `fullM2LetGraphAliasPresentationComplete` が `visible graph presentation` を構成する。
2. `fullM2LetSupportedAssemblyBridge` が `whole-let` 証明書を構成する。
3. `fullM2CoherenceComplete` が通常構文の帰納法と `let` の場合を統合する。
4. `FullM2Replay` が任意の関係的導出を実行可能な導出へ再生する。
5. `wellFormedElaborationPrincipalityComplete` が完全性と主要性をまとめる。

ここで *coherence* は、同じ `source` に対する異なる正しい導出が、受理と主要結果について整合することをいう。*replay* は、関係的導出を入力として、公開推論器が実際に選ぶ `closure` を使った導出を再構成することである。

6.2 公開 API

最終的に次の定理が公開される。

定理	初学者向けの読み方
<code>Typing.infer_isSome</code>	独立した型付けがあれば、公開推論は失敗しない。
<code>Inference.typable_iff_infer_isSome</code>	<code>well-formed signature</code> の下で、型が付くことと推論成功が同値。
<code>Inference.typableDecidable</code>	型が付くかどうかを公開推論で決定できる。
<code>Inference.infer_success_principalResult</code>	推論が返した型は、すべての型付け結果の主要型である。
<code>Inference.infer_principalResult</code>	関係側の主要な型付けから、公開推論の主要結果を得られる。
<code>PrincipalTyping.finiteRenamingEq</code>	二つの主要代表は、有限個の変数名変更を除いて一致する。

`Typing.infer_isSome` と `finiteRenamingEq` は `signature` の `well-formedness` を仮定しない。推論成功から関係的型付けを得る健全性まで使う API は、`Signature.WellFormed` を明示的に受け取る。signature が `well-formed` であるとは、名前が重複せず、各型 `scheme` が外部の未定義変数を含まず、`constructor` の引数数や結果の `data` 型が宣言表と一致することである。

6.3 負例にも完全性が効く

`M2Regression.cutRejectsBackflow` は、`let` 右辺を閉じた後に、`body` から互換でない要求を右辺へ逆流させようとする例である。実行可能推論が `none` を返すだけなら、推論器が不完全で見逃した可能性が残る。M2 完全性により、もし独立した `Typing` が存在すれば推論は成功するはずである。実際には失敗するので、`cutRejectsBackflow_not_typable` として非型付けまで証明できる。

第 7 章

M3 から M5 までの現在地

7.1 M3：宣言と通常と呼出し

M3 では Bool/List の data 型, data constructor, pattern constructor, primitive, 有限 signature を追加した. data constructor は `nil` や `cons` のように data 値を作る名前であり, primitive は `add` や `map` のように実装があらかじめ与えられた組込み演算である. constructor と primitive の引数処理は, 通常の間数適用 `Generated.fromApp` を左から畳み込む. したがって M2 で証明した通常 application の構造を再利用できる.

`add`, `append`, `member`, `deleteFirst`, `map` について, 実装が正しいものとして固定する標準の型 scheme (canonical scheme), `ifE`, 名前・引数数・型不一致の正負回帰は実装済みである. M3 が表で「一部」なのは, M4 の pattern/matcher と合わせた論文 listing 全体の静的回帰が残るためである.

7.2 M4：pattern と matcher

source 構文は `fixE`, `matcher`, `matchAll`, `matchFirst`, `value pattern`, `pattern constructor`, `conjunction pattern`, `pattern function application` まで持つ. `conjunction pattern` とは, 同じ対象へ二つの pattern を左から右の順に適用する pattern である. M4 の実行可能 elaboration と関係の elaboration は同じ構文を再帰的に処理し, 成功から `M4.Typing` を得る健全性を証明済みである. ここで elaboration は source 式から型制約を生成する処理である. 実行可能版は Lean の関数として結果を計算し, 関係版は「この生成結果が規則から導ける」という命題として同じ処理を表す.

fresh 変数の由来を追う support provenance も証明している. support とは, 型や制約に実際に現れる変数の有限集合である. provenance は「由来」という意味であり, ここでは各変数がもとの context にあったか, 開始 supply から終了 supply までの間に新しく割り当てられたことを表す. well-formed signature, つまり同じ名前の宣言が重複せず, 各 scheme が正しく閉じている宣言表の下では, 最終的な相互再帰関係 `ElaboratesFuel` と, 公開関係 `M4.Elaborates` を含む全 M4 構文についてこの性質を証明済みである. 公開された supply 増加と由来証明をまとめた `M4.supplyAndSupport` も完成した. これは生成変数の管理が正しいという結果であり, M4 全域の完全性が得られたという意味ではない.

一方, M2–M3 で完了した「型が付くなら必ず推論成功」と「返却型の主要性」は M4 全域では未証明である. `matcher literal` を構文だけから検査する処理 (static check), `fix` 直下 `matcher` の型形

状, clause 内部の slot 要求, および `let` での任意 closure 選択を同時に扱う必要があるため, 独立した大きな課題である.

7.3 M5: 評価と型安全性

M5 にはプログラム実行中に実際に得られる値 (runtime value), 関数本体と定義時の環境を組にした関数 closure, matcher 本体と環境を組にした matcher closure, 候補を記録順に奥まで調べる深さ優先探索, `matchAll` と `matchFirst` の関係の評価と fuel 付き評価器がある. fuel は再帰計算を進められる回数の上限である. 有限な成功導出について, 実行可能評価の健全性, 完全性, fuel 単調性を証明している. fuel 単調性とは, ある fuel 量で成功した計算が, より大きい fuel 量でも同じ結果で成功することである.

conjunction pattern は組込み規則 A-AND で実行する. A-AND は一つの照合課題を, 同じ matcher と同じ対象を持つ左 pattern, 右 pattern の二課題へこの順で置き換える. 左側で得た binding が右側の value pattern から見えるため, 静的型付けと実行の変数順が一致する.

7.4 一般 pattern function と MNode

pattern function は pattern を引数に取り, 別の pattern を組み立てる機能である. 本体内で一時的に束縛した変数は外側の `matchAll` へ見せてはならない一方, 引数として渡された pattern が作る束縛は外側へ返す必要がある. この二つを区別するため, 実行時には MNode を使う. MNode は, pattern function 本体専用の照合課題, 環境, private binding (本体の中だけで使う束縛) を持つ隔離された探索 node である.

具体例は Paper 2 の `pair` である.

```
def pattern pair (first, rest) :=
  ($private & ~first) :: #private :: ~rest
```

`$private` は最初の要素を本体内だけで束縛し, `#private` が次の要素と同じかを検査する. `~first` と `~rest` は, 呼出し側から渡された第一・第二引数 pattern を表す. MNode の処理は次の順である.

1. `pair q1 q2` を matcher へ渡す前に, `pair` 本体を持つ MNode へ展開する.
2. 本体の通常照合は MNode 内部で進め, `private` の束縛も内部に保存する.
3. `~first` または `~rest` へ着いたら, 対応する実引数 pattern を外側の探索へ戻す.
4. 本体が終わったら MNode 内部の束縛を捨て, 実引数 pattern が作った外側の束縛だけを残す.

また, pattern function 適用は user-defined matcher の catch-all clause より先に処理する. catch-all clause とは, それ以前の clause に一致しなかった pattern を一般的に受け取る最後の clause である. 先に MNode へ展開しなければ, `pair` そのものが catch-all に捕まり, 本体が実行されないからである.

正確な 7-clause source multiset matcher を使う回帰では,

```
matchAll [1,2,1,3] as multiset integer with
  $outer :: pair $inner _ -> [outer, inner]
```

の照合結果に対応する binding 列として,

$$[[1, 2], [1, 2], [1, 3], [1, 3]]$$

を外側, 内側の順で並べ, 順序と重複を保って得る全 `matchAll` 式を Lean kernel で確認している. 同じ探索を使う `matchFirst` が先頭の `[1, 2]` を返すことも固定した. `private` は各結果に含まれない. pattern function 本体の静的な検査証明も, 保存 scheme の標準的な一つの具体化について, 生成された等式と checking 条件を実際に満たす代入と, 代入後の結果型の一致を保持する. これは, 全具体化に対する interface 一致や主要性の証明ではない. さらに, source/runtime 定義表の双方向対応を `PatternFunctionDefinitions.Agree` で表す. これは, runtime 表の各本体が上記の標準的な具体化に対する source 側の検査証拠を持ち, source 側の各宣言に runtime 実装があるという条件である. したがって, runtime だけにある未検査本体と, 実装のない source 宣言の両方を除ける. `pair` の定義表がこの条件を満たすことも証明済みである.

MNode 探索は `evalCheckedPatternFunctionNodesFuel` という公開 fuel 評価器にも接続した. この評価器は定義表と上記の双方向対応の証明を受け取り, 全 `Expr` を再帰的に評価し, `matchAll` と `matchFirst` では MNode 探索を使う. 成功した `matchAll` の探索部分について, 実行可能な一步関数を使う帰納的な順序付き深さ優先導出も証明済みである. ただし評価器全体に対応する独立な関係的評価 `Eval` と, MNode を含む型安全性はまだ証明していない.

型安全性は整数, 真偽値, List, tuple, 型付き環境, 通常・再帰 closure, 任意の型付き関数位置を持つ application, `map`, および明示的な callback 仮定の下での built-in matcher 断片までである. callback とは, 処理本体を引数として受け取り, 必要な時点で呼び出す関数である. built-in matcher とは, 利用者が source で定義するのではなく, 実行器に直接実装された matcher である. ここで「任意の型付き関数位置」とは, application の左側が構文上直接の lambda や `fixE` でなくても, 実行時型付けで関数型を持てばよいという意味である. 例えば条件分岐が返した closure の適用も扱える. source の多相型付けからこの関数断片を再構成する橋, user-defined matcher clause, 一般 pattern function の MNode を同じ型安全性契約へ接続する作業が残る. ここで保存とは, 実行の一步後も型が保たれることであり, 進行とは, 型の付いた実行可能状態が結果でなければ次の一步を進められることである. 残作業はこの保存・進行の形を持つが, 多相 `let` の scheme と runtime 環境を結ぶ設計を含むため, 小さな機械作業ではない.

第 8 章

multisetWithListArgument を理解する

8.1 Paper 1 の multiset 定義との対応

現行 source AST, つまり source 式の構文を木として表すデータ型には, プログラム最外部 (top-level) の利用者定義値を名前で参照する module 環境がない. module 環境とは, 名前からその定義済みの値を引く表である. 通常の値参照は de Bruijn index で表す. de Bruijn index とは, 変数名の代わりに「最も内側の binder から何個外か」を自然数で表す方法である.

Paper 1 の概略

```
def multiset m := matcher { ... uses list m ... }
```

では, list は既に定義済みの library matcher である. Lean fixture では multisetDefinition の外側環境に list が一つあるものとして, 内部から index 2 で参照する. index 0 は fixE の引数 m , index 1 は再帰的な自己 multiset, index 2 が外側の list である.

8.2 三つの式の役割

```
multisetDefinition
  = fixE (matcher multisetClauses)

multisetWithListArgument
  = lambda list. multisetDefinition

closedMultisetDefinition
  = multisetWithListArgument listMatcherDefinition
```

multisetDefinition 自体が, Paper 1 の再帰的 `def multiset m := ...` に対応する. fixE を適用すると, 変数から実行中の値を引く表 (実行時環境) は `argument :: self :: definitionEnvironment` となる.

multisetWithListArgument は Paper 1 に表示される別のアルゴリズムではない. 自由依存である list を lambda 引数へ持ち上げた, closure conversion 風の補助式である. closure conversion とは, 関数が外側から参照する自由な値を, 明示的な環境や引数として渡す変換である.

closedMultisetDefinition は source で定義した listMatcherDefinition を実際に渡し, 空の

source context から使える閉じた式にする。閉じた式とは、外側の変数を参照せず、それだけで評価できる式である。静的確認だけなら、`multisetDefinition` を既知の list scheme を持つ context で調べてもよい。end-to-end の実行回帰では隠れた primitive 値を使わず、空 context・空の実行時環境 (runtime environment) から始めるため、この明示的な適用を置いている。

8.3 なぜ lambda 版単独は現在拒否されるのか

`multisetWithListArgument` の制約生成自体は成功し、hard 制約も解ける。失敗は残った checking 要求の解決で起きる。外側 lambda 引数 `list` の型は概略

$$\text{Slot } \chi \alpha \rightarrow \beta$$

まで hard から分かるが、返り型 β が matcher producer であることは hard だけから決まらない。

clause 内には概略

$$\text{Prod}[\beta, \text{Matcher}(\text{List } \chi)(\text{List } \alpha)] \leftarrow \text{product-slot}$$

という要求が残る。product から product-slot への特殊変換を選ぶには、product の全成分が構文上すでに matcher だと分かる必要がある。 β が変数のままなので、resolver は普通の等式へ進み、最外 constructor の不一致で失敗する。

型が存在しないのではない

β を適切な matcher 型へ具体化すれば、すべての checking 要求を満たす意味的な解はある。期待される全体型は概略

$$(\text{Slot } \chi \alpha \rightarrow \text{Matcher}(\text{List } \chi)(\text{List } \alpha)) \rightarrow \text{Slot } \chi \alpha \rightarrow \text{Matcher}(\text{List } \chi)(\text{List } \alpha)$$

である。しかしその形を選ぶには、hard の主要解に現れていない matcher 外形を β へ先回りして与える必要がある。現在の branch-stable/no-guess 仕様はこの推測をしない。

`closedMultisetDefinition` では、実引数 `listMatcherDefinition` の既知の型が外側 lambda の domain へ hard な情報を与え、 β が matcher だと確定する。その後なら同じ product 要求を特殊変換として分類でき、推論が成功する。

この現象は偶然の M4 実装バグではなく、`GeneratedSemanticAcceptanceRegression` にある M1 の最小境界の大規模な実例である。「完全に具体化すれば意味的に checking 可能」と「hard の主要解で分岐を固定した公開手続きが受理する」は同じではない。

8.4 設計上の選択肢

この高階 matcher constructor を source 言語でどこまで受理したいかにより、選択肢が変わる。

1. 既知型 context または明示注釈を使う。no-guess を保つ最も安全な案である。
2. matcher 位置から期待 slot を逆向きに伝える。双方向型付けに近い規則を追加する。どの構文位置が十分な根拠になるかを明示する必要がある。

3. **checking 分岐を探索する**. より多くの意味的解を受理できるが, branch-stable 設計と主要性の証明を大きく変更する.

現状では, Paper 1 の `multiset` 本体そのものが無意味なのではない. 名前付き top-level 環境を持たない fixture を閉じるための外側 `lambda` が, 高階 `matcher constructor` に対する既知の `no-guess` 境界を露出している, という位置づけである.

8.5 別問題: kernel で大きな推論を計算すること

大きい `matcher` の検証例 (fixture) では `M4.infer` の実行結果を `#eval` で確認できても, Lean kernel の定義簡約だけで同じ等式を `rfl` により証明することが難しい. Lean kernel とは, 作られた証明が Lean の規則に従うかを最後に確認する, 小さな検査器である. 測度に基づく再帰で定義した `pattern` 検査や単一化が簡約を遮るためである.

これは前節の `branch-stable` 拒否とは別問題である. 改善するなら, 既存の `fuel` 構造再帰 `M4.elaborateFuel` を `solver` まで引数化し, `pattern` 検査・`direct-self` 検査・単一化にも計算可能な `fuel` 版を用意する. その成功を公開版へ輸送する定理を証明した後, 小さい `catch-all`, `list matcher`, `closed multiset` の順で `kernel` 回帰を固定するのが自然である.

第 9 章

コードを読むための案内

9.1 最初に読むファイル

ファイル	読める内容
TypePM/Constraints.lean	Equation, CheckObligation, Generated の最小定義.
TypePM/Source/Syntax.lean	M4 までの最終 source AST.
TypePM/Source/Elaboration.lean	実行可能生成と独立した Elaborates, fromApp, fromLet.
TypePM/BlockClosure.lean	右辺 block を閉じる PrincipalBlockClosure.
TypePM/ContextInterface.lean	外側 context への有限な interface equations.
TypePM/Source/M2Regression.lean	多相 identity, 明示的 let, full-cut 負例.
TypePM/Source/ GlobalRenamingCounterexample.lean	一つの大域 rename では足りない具体的反例.
TypePM/ EntailedPendingTransport.lean	pending を hard 理論の全解の下で比較し, 受理を輸送する層.
TypePM/Source/ EntailedAlignment.lean	source block の意味的な比較証明書.
TypePM/Source/ WholeLetEntailedAssembly.lean	interface と pull back した body alias の whole-let 合成.
TypePM/Source/ FullM2GraphAliasPresentation.lean	visible closure graph の source 由来構成.
TypePM/Source/FullM2Coherence.lean	通常構文と let の場合を統合する帰納法.
TypePM/Source/FullM2Replay.lean	関係的導出から実行可能導出への再生.

ファイル	読める内容
TypePM/Source/ FullM2Completion.lean	完了した M2 の公開入口.
TypePM/Source/Paper1Programs. lean	list/multiset matcher の完全な source fixture.
TypePM/ GeneratedSemanticAcceptance Regression.lean	branch-stable 受理と意味的解の差を示す最小例.

9.2 検証コマンド

Lean 全体はリポジトリ直下で次のように検証する.

```
lake build
git diff --check
```

追加公理への依存は TypePM/AxiomAudit.lean で検査する. M2 の公開入口は Lean 標準の `propext`, `Classical.choice`, `Quot.sound` 以外の追加公理に依存しない.

本書は `tex` ディレクトリで次のように生成する.

```
make
```

出力は `type-pm-mech2-guide.pdf` である.

第 10 章

まとめと再開地点

10.1 M2 について覚えておく四文

1. `let` 右辺は full-cut で先に閉じ、右辺 pending を body へ逆流させない.
2. 任意の主要 closure は意味的には同じでも、代表変数と interface が文字どおり違い得る.
3. body は future 名を固定する有限 rename で同じ context へ運び、outer interface は別の visible alias graph で共通化する.
4. pending は文字列一致でなく、hard を解くすべての代入の下で同じかどうかを比較する.

この四層を well-formed supply による freshness が支え、公開推論の完全性・受理同値・主要性へつながる.

10.2 次の作業は M2 ではなく M4/M5

M2 の ill-formed 開始 supply は仕様外であり、完了のために追加証明する項目ではない. 次の静的な主作業は、大きい Paper 1 の検証例を Lean kernel で計算可能な形にし、正例の正確な `M4.infer` 結果と `M4.Typeing`, 負例の非型付けを固定することである.

fresh 変数の由来証明は公開 `M4.Elaborates` まで完成したため、次は M4 全域の完全性・主要性を別計画として調べる. M5 では source の多相 `let` から、すでに任意の型付き関数位置と `map` を扱える実行時型付けへの橋を作り、user-defined matcher clause と pattern function の MNode を一般の実行時型付けと、型の不整合による停止が決して起きないという性質 (no-stuck) へ接続する.

付録 A

現行実装の型規則一覧

A.1 この一覧の読み方

現行実装の型付けは、式から最終型を一度に決めるのではなく、次の二段階に分かれる。

1. **制約生成**：式を左から右へ調べ、**target**, **hard**, **pending** を作る。
2. **制約を閉じる処理**：hard を単一化し、pending の checking 規則を分類して解き、最終的な主要型を得る。ここで主要型とは、ほかの解を代入によって得られる最も一般的な型である。

したがって、本付録で中心になるのは通常の「 e の型は τ である」という判断より一段手前の**制約生成判断**である。次の記号を使う。

$$\Sigma; \Gamma; s \vdash e \Longrightarrow \langle t; H; P \rangle; s'$$

これは「宣言表 Σ と context Γ の下で、開始 supply s から式 e を調べると、target t , hard 列 H , pending 列 P を生成し、次の未使用番号が s' になる」という意味である。宣言表とは data constructor, primitive, pattern constructor などの型を登録した表である。 $\langle t; H; P \rangle$ を以下では生成 block と呼ぶ。 $H_1 ++ H_2$ は列を順に連結する記号である。

型変数と capability 変数には別々の番号がある。 s から新しく取る型変数を α_s , capability 変数を κ_s と略記する。実装上の s はこの二種類の次番号を持つ組である。「fresh な変数」とは、それまで使われていない新しい番号の変数を指す。

規則の上と下

横線の上は、その規則を使うために先に確かめる条件である。横線の下は、条件がそろったときに導ける結論である。条件が複数ある場合は、source 順に左から右へ調べる。本付録では読みやすさのため、supply の単純な加算や列要素の再帰的な連結は一部を文章で表す。省略された検査を意味しない。

A.2 生成 block を組み立てる共通操作

以下の四つを押さえると、個々の規則を短く読める。

操作	生成する block
lambda	body が $B = \langle t_B; H_B; P_B \rangle$ なら, $\text{Lam}(d, B) = \langle d \rightarrow t_B; H_B; P_B \rangle$. 引数型 d を結果型の前へ付ける.
application	関数 $F = \langle t_F; H_F; P_F \rangle$ と引数 $A = \langle t_A; H_A; P_A \rangle$ に対して $\text{App}(F, A; d, r) = \langle r; H_F ++ H_A ++ [t_F = d \rightarrow r]; P_F ++ P_A ++ [t_A \leftarrow d] \rangle.$
checked expression	d は引数位置の型, r は適用結果の型で, どちらも fresh に取る. $G = \langle t; H; P \rangle$ を expected 型 u として使う場合, $\text{Checked}(G, u) = \langle H; P ++ [t \leftarrow u] \rangle$. 結果型を返さず, checking 要求を一件追加する補助形である.
let	body が $B = \langle t_B; H_B; P_B \rangle$, 右辺を閉じた効果が等式列 I なら, $\text{Let}(I, B) = \langle t_B; I ++ H_B; P_B \rangle$. 右辺自身の pending は閉じる処理で解決済みなので外へ出さない.

A.3 M2–M3 の式の規則

ここで scheme は量化変数を持てる多相型であり, instantiate は量化変数を fresh な自由変数へ置き換える操作である. 以下の規則は TypePM/Source/Elaboration.lean の Elaborates, ElaboratesItems, ElaboratesCall に対応する.

A.3.1 変数, 整数, something

$$\frac{\Gamma[i] = \sigma \quad \text{instantiate}(\sigma, s) = (\tau, s')}{\Sigma; \Gamma; s \vdash \text{var } i \Longrightarrow \langle \tau; []; [] \rangle; s'} \text{VAR}$$

context の i 番目にある scheme σ をその使用場所で instantiate する. したがって多相変数は使用するたびに別の fresh 変数を得る.

$$\frac{}{\Sigma; \Gamma; s \vdash n \Longrightarrow \langle \text{Int}; []; [] \rangle; s} \text{INT}$$

$$\frac{}{\Sigma; \Gamma; s \vdash \text{something} \Longrightarrow \langle \text{Matcher Any } \alpha_s; []; [] \rangle; s'} \text{SOMETHING}$$

整数は hard も pending も作らない. something は任意の対象型 α_s に対する matcher producer を作り, 型変数の一つ消費する.

A.3.2 lambda, application, tuple

$$\frac{\Sigma; (\alpha_s :: \Gamma); s_1 \vdash e \Longrightarrow B; s'}{\Sigma; \Gamma; s \vdash \lambda e \Longrightarrow \text{Lam}(\alpha_s, B); s'} \text{LAM}$$

s_1 は α_s を一つ予約した後の supply である. $\alpha_s :: \Gamma$ は, lambda の引数型を context の先頭へ追加することを表す.

$$\frac{\Sigma; \Gamma; s \vdash f \Longrightarrow F; s_1 \quad \Sigma; \Gamma; s_1 \vdash a \Longrightarrow A; s_2}{\Sigma; \Gamma; s \vdash f a \Longrightarrow \text{App}(F, A; d, r); s_3} \text{APP}$$

d, r は s_2 から取る fresh な型変数である. 関数の外形 $t_F = d \rightarrow r$ は hard へ入るが, 引数の使用可能性 $t_A \Leftarrow d$ は pending へ入る. 第 2.3 節の具体例はこの規則の二通りの解決である.

$$\frac{\Sigma; \Gamma; s \vdash [e_1, \dots, e_n] \Longrightarrow [(t_1, H_1, P_1), \dots, (t_n, H_n, P_n)]; s'}{\Sigma; \Gamma; s \vdash (e_1, \dots, e_n) \Longrightarrow \langle \text{Prod}[t_1, \dots, t_n]; H_1 ++ \dots ++ H_n; P_1 ++ \dots ++ P_n \rangle; s'} \text{TUPLE}$$

要素列は左から右へ調べる. 空 tuple では型が $\text{Prod}[]$, hard と pending が空になる.

A.3.3 多相 let

$$\frac{\Sigma; \Gamma; s \vdash v \Longrightarrow V; s_v, \quad C \text{は} V \text{の吸収的な主要 closure,} \quad \Sigma; (\text{gen}(C(t_V), C(\Gamma)) :: C(\Gamma)); s_b \vdash e \Longrightarrow B; s'}{\Sigma; \Gamma; s \vdash \text{let } v \text{ in } e \Longrightarrow \text{Let}(\text{interface}(\Gamma, C), B); s'} \text{LET}$$

v が let の右辺, e が body である. closure C は右辺 block を主要に解いた結果で, $C(t_V)$ は右辺 target へその代入を適用した型, $C(\Gamma)$ は context へ同じ代入を適用したものを表す. gen は contextに残っていない変数を量化して scheme を作る一般化である. s_b は右辺終了 supply と閉じた context が要求する開始番号の大きい方である. $\text{interface}(\Gamma, C)$ は, 右辺を閉じた影響のうち外側 context から観測できる有限な等式列である.

A.3.4 data constructor, primitive, 条件式

data constructor と primitive は, 宣言表から閉じた scheme を引き, instantiate した関数型へ引数一つずつ APP と同じ形で適用する.

$$\frac{\Sigma(c) = \sigma \quad |\vec{e}| = \text{arity}(\sigma) \quad \sigma \text{は閉じている} \quad \text{Call}(\text{instantiate}(\sigma, s), \vec{e}) = G; s'}{\Sigma; \Gamma; s \vdash c(\vec{e}) \Longrightarrow G; s'} \text{CTOR}$$

$$\frac{\Sigma(op) = \sigma \quad |\vec{e}| = \text{arity}(\sigma) \quad \sigma \text{は閉じている} \quad \text{Call}(\text{instantiate}(\sigma, s), \vec{e}) = G; s'}{\Sigma; \Gamma; s \vdash op(\vec{e}) \Longrightarrow G; s'} \text{PRIM}$$

閉じた scheme とは, 外から与えられていない自由変数を含まない scheme である. `if c then t else e` は, 組込み scheme $\text{Bool} \rightarrow \alpha \rightarrow \alpha \rightarrow \alpha$ を instantiate し, $[c, t, e]$ を同じ Call で処理する If 規則である. これにより条件は Bool, 二枝は同じ型になる.

A.4 M4 の user pattern 規則

user pattern は, 値に照合する `matchAll` や `matchFirst` の pattern である. 次の判断を使う.

$$\Sigma; \Gamma; \Phi; \Delta; s \vdash_p p \Longrightarrow \langle \delta; \Delta'; H; P \rangle; s'$$

Φ は pattern function へ渡された pattern 引数の型列, Δ はその pattern の左側ですでに束縛された通常変数の型列である. $\delta = (\kappa \triangleright \tau)$ は dual であり, 「capability κ を使って型 τ の値へ照合する pattern」を表す. Δ' はこの pattern までに束縛された変数を source 順に並べた結果である.

pattern	結果	説明
variable	$\kappa_s \triangleright \alpha_s, \Delta' = \Delta ++ [\alpha_s]$	fresh な対象型と capability を作り, 対象値を新しい変数として束縛する.
wildcard	$\kappa_s \triangleright \alpha_s, \Delta' = \Delta$	同じ fresh dual を作るが, 値を束縛しない.
value pattern $\#e$	$\kappa_s \triangleright t_e$	e を $\Delta ++ \Gamma$ で調べ, その hard と pending を引き継ぐ. 新しい capability だけを割り当てる.
constructor $c(p_1, \dots, p_n)$	宣言済み dual scheme の result	constructor を lookup し, arity を確認する. 各 field pattern の target と capability を, instantiate した field dual へ hard 等式でそろえる.
tuple $(p_1, \dots, p_n)$	$\text{Prod}[\kappa_1, \dots, \kappa_n]$ $\text{Prod}[\tau_1, \dots, \tau_n]$	$\triangleright$ field pattern を左から右へ処理し, capability と target をそれぞれ product にする.
conjunction $p \& q$	δ_p と, q までの binding 列	同じ対象へ p, q の順で照合する. p の binding を追加してから q を調べ, 両側の target 型と capability を hard 等式で同じにする.
embedded parameter	$\Phi[i]$	pattern function の i 番目の引数 dual をそのまま使う. fresh 変数や束縛を作らない.
pattern function $f(\vec{p})$	宣言済み dual scheme の result	frozen signature から f の scheme だけを instantiate する. 本体を再検査せず, field dual を scheme の引数 dual へ hard 等式でそろえる.

A.5 M4 で追加された式の規則

M4 の公開関係 `M4.Elaborates` は, 前節までの式規則を再帰的に使いながら, 次の四形式を追加する. 内部では構文の大きさより大きい自然数を fuel として持つ. fuel とは再帰呼出しの残り回数を表す数で, 公開判断は「十分な fuel が一つ存在する」とだけ要求するため, program の型には現れない.

A.5.1 単項再帰 fix

$$\frac{\text{DirectSelf}(b) \quad \Sigma; (d, d \rightarrow r, \Gamma); s_b \vdash b \Longrightarrow \langle t_b; H_b; P_b \rangle; s'}{\Sigma; \Gamma; s \vdash \text{fix } b \Longrightarrow \langle d \rightarrow r; H_b ++ [t_b = r]; P_b \rangle; s'} \text{FIX}$$

body context の先頭は引数 d 、次が再帰関数自身 $d \rightarrow r$ である。DirectSelf は、自己参照を別名に保存したり高階の値として逃がしたりせず、許可された直接呼出しだけを使う構文検査である。body が構文上 matcher literal なら、 d を matcher slot、 r を matcher producer の形へ最初から固定する。これは matcher という明示構文から決まる形であり、pending から型の外形を推測する操作ではない。

A.5.2 matcher literal

$$\frac{\text{staticChecks}(\Sigma, \text{clauses}) = \text{true} \quad \Sigma; \Gamma; \alpha_s; s_1 \vdash_{\text{clauses}} \text{clauses} \Longrightarrow (E; H; P); s'}{\Sigma; \Gamma; s \vdash \text{matcher}\{\text{clauses}\} \Longrightarrow \langle \text{Matcher } \kappa_s \alpha_s; \text{evidenceEq}(\kappa_s, E) ++ H; P \rangle; s'} \text{MATCHER}$$

α_s は matcher が調べる対象値の型、 κ_s は matcher 全体の capability である。staticChecks は、constructor の arity、変数の重複禁止、catch-all の位置、data arm の網羅性など、型等式を解く前に判定できる構文条件をまとめた検査である。 E は各 clause が示した分解能力の証拠で、evidenceEq がそれらを全体 capability へ hard 等式として接続する。

一つの clause は次の順で調べる。

1. pattern-pattern header を調べ、hole dual 列と capture 型列を得る。hole は次に必要な matcher の場所、capture は $\#\$x$ のように対象値全体を式で参照する束縛である。
2. next-matcher 式を、hole が 0 個なら unit product、1 個ならその slot、2 個以上なら slot の tuple として checking する。
3. 各 data-pattern arm を対象型 α_s に対して調べ、arm body を $\text{List}(\text{holeProductTarget}(\text{holes}))$ として checking する。

pattern-pattern header の hole は、与えられた対象型と fresh な capability を一組の hole dual にする。constructor header は宣言済み pattern-constructor scheme を instantiate し、field を再帰的に調べる。以下が header と arm pattern の constructor 別一覧である。

種類	constructor	生成内容
pattern-pattern	hole	与えられた対象型 τ と fresh な κ_s から $[\kappa_s \triangleright \tau]$ という hole 列を作る。expected capability があれば、 κ_s との hard 等式も追加する。
pattern-pattern	wildcard	hole も capture も hard も作らない。

種類	constructor	生成内容
pattern-pattern	capture	与えられた対象型 τ を capture 列へ一つ追加する. hole は作らない.
pattern-pattern	constructor	宣言済み scheme を instantiate し, arity を確認して field を再帰的に調べる. result target を与えられた対象型へ hard 等式でそろえ, result capability を分解能力の証拠として返す.
data-pattern	variable	与えられた対象型 τ を arm body 用の束縛列へ追加する.
data-pattern	wildcard	束縛も hard も作らない.
data-pattern	constructor	data-constructor scheme を instantiate して関数型を field 型列と result 型へ分ける. result 型を与えられた対象型へ hard 等式でそろえ, field を再帰的に調べる.
data-pattern	tuple	要素数だけ fresh な field 型を作り, 与えられた対象型をその product へ hard 等式でそろえる. 各要素を対応する field 型に対して再帰的に調べる.

A.5.3 全結果を返す matchAll

target 式 e_t , matcher 式 e_m , user pattern p , body e_b の生成結果をそれぞれ T, M, Q, B とする. Q の dual を $\kappa_p \triangleright \tau_p$ とすると,

$$\frac{\begin{array}{l} \Sigma; \Gamma; s \vdash e_t \Longrightarrow T; s_1, \quad \Sigma; \Gamma; []; []; s_1 \vdash_p p \Longrightarrow Q; s_2, \\ \Sigma; \Gamma; s_2 \vdash e_m \Longrightarrow M; s_3, \quad \Sigma; (\Delta_Q \dashv\vdash \Gamma); s_3 \vdash e_b \Longrightarrow B; s' \end{array}}{\Sigma; \Gamma; s \vdash \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e_b \Longrightarrow G; s'} \text{ MATCHALL}$$

ここで G の target は $\text{List}(t_B)$ である. hard には $\tau_p = t_T$ を追加し, pending には $t_M \leftarrow \text{Slot } \kappa_p t_T$ を追加する. つまり pattern の対象型は実際の target 値と等しくし, matcher 式はその pattern が要求する slot として使えるかを後で checking する. Δ_Q は pattern が束縛した変数列で, body context の先頭へ source 順に追加する.

A.5.4 最初の結果を返す matchFirst

$$\frac{\begin{array}{c} \text{armsExhaustive}(\text{arms}) = \text{true}, \\ \Sigma; \Gamma; s \vdash e_t \Longrightarrow T; s_1, \quad \Sigma; \Gamma; s_1 \vdash e_m \Longrightarrow M; s_2, \\ \Sigma; \Gamma; t_T; t_M; s_2 \vdash_{\text{arms}} \text{arms} \Longrightarrow A; s' \end{array}}{\Sigma; \Gamma; s \vdash \text{matchFirst } e_t \text{ as } e_m \text{ with } \text{arms} \Longrightarrow \text{fromMatchFirst}(T, M, A); s'} \text{ MATCHFIRST}$$

arm 列は空であってはならず、最後の pattern は必ず一致すると構文的に分かる形でなければならない。各 arm では pattern target を t_T へ hard 等式でそろえ、matcher t_M を pattern capability の slot として pending checking する。最初の arm body の target を全体結果型とし、後続 body の target はその型へ hard 等式でそろえる。matchAll と違い、結果へ List を付けない。

A.6 最終的な Typing 規則

制約生成に成功しただけでは、hard と pending が解けるとは限らない。現行の最終判断は次の二段階である。

$$\frac{\begin{array}{c} \Sigma; \Gamma; \text{initialSupply}(\Gamma) \vdash e \Longrightarrow G; s', \\ C \text{ は } G \text{ の吸収的な主要 closure,} \quad \pi = C(t_G) \end{array}}{\Sigma; \Gamma \vdash_{\text{principal}} e : \pi} \text{ PRINCIPAL TYPING}$$

$$\frac{\Sigma; \Gamma \vdash_{\text{principal}} e : \pi \quad \tau = S(\pi)}{\Sigma; \Gamma \vdash e : \tau} \text{ TYPING}$$

S は型変数への任意の代入である。したがって Typing は主要型 π そのものだけでなく、その型変数をさらに具体化して得られる型 τ も認める。M2–M3 では Source.Typing, M4 では Source.M4.Typing がこの同じ二段階の形を持つ。

一覧の範囲

ここに載せたのは現行 source AST の型付け・制約生成規則である。hard の単一化、pending の promotion と residual checking, closure の主要性を証明する規則群は、生成された block を解く側の関係であり、source 構文の型規則とは分けている。また M4 の健全性は実装済みだが、M4 全体について「宣言的に型が付けば公開推論が必ず成功する」という完全性と、すべての導出間の主要性は未完である。この一覧は未証明の完全性を仮定しない。

付録 B

用語集

absorbing closure	後続の解を合成しても、選んだ主要代表が正規化された形で保たれる closure.
branch-stable acceptance	hard の主要解で checking 分岐を固定し、後から分岐選択をやり直さない受理概念.
capability	matcher が入力値のどの形を観察できるかを表す型情報.
closure	生成 block の hard と残余 checking を主要に解く代入と結果をまとめたもの. 実行時関数 closure とは別.
coherence	同じ source に対する異なる正しい導出が、受理と主要結果について整合する性質.
entailed	ある hard 理論を満たすすべての代入の下で必ず成り立つこと.
frame	生成済み制約 block を穴へ入れる周囲の制約構造.
freshness	新しく割り当てた変数名が、既存の support と衝突しない性質.
full-cut	let 右辺の制約 block を body より先に完全に閉じる方式.
hard	checking 変換の種類に依存せず必ず満たす型・capability 等式.
interface equations	閉じた let 右辺が外側 context の自由変数へ与えた影響を表す有限な等式列.
matcher producer	実際に作られた matcher 値の型.
matcher slot	matcher を使用する場所が matcher へ課す要求型.
pending	source 型を expected 型として使えるか, hard を解いた後で分類する checking 要求.
principal	ほかのすべての解が代入を通して得られるという意味で最も一般的であること.
replay	関係の導出から、公開推論器が実際に選ぶ closure を用いた導出を再構成すること.
scheme	量化された型変数を持つ多相型. 現行 M2 では量化変数を位置で表す.
supply	新しい型変数名と capability 変数名を割り当てる二つの自然数カウンタ.
support	型, 制約, 代入などに実際に現れる有限な変数集合.
target	制約生成直後の生の結果型. 最終代入の適用前なので型変数を含んでよい.
terminal audit	推論完了後に履歴や状態を再検査する旧方式. 現行 Typing の定義には含まれない.

visible closure graph	二 closure の有限な変数対応のうち，外側 closed context から観測できる移動辺だけを集めた証明用構造.
well-formed supply	context に既に現れる全自由変数より後から fresh 名を割り当てる開始条件.

付録 C

旧説明から現行説明への対応

旧資料で見かける語	現行 type-pm-mech2 での扱い
SourceTyping	使用しない. 現行は実行可能推論器と独立した <code>Source.Typing</code> を定義する.
inferRaw	旧 API. 現行 source 側は <code>elaborate</code> , <code>infer</code> , M4 側は <code>M4.infer</code> を使う.
wBridgeCheck	旧 terminal audit の一部. 現行の宣言的型付け定義には持ち込まない.
TypingInvariant	旧推論履歴の検査. 現行 M2 完全性は <code>entailed alignment</code> , <code>closure alignment</code> , <code>visible graph</code> で証明する.
PublicTheorems.lean	旧リポジトリの索引. 現行の公開 M2 入口は <code>Source/FullM2Completion.lean</code> .

版の基準

本書は 2026 年 8 月 20 日に, `type-pm-mech2` の現行 `main` の実装と公開監査項目を基準に更新した. 文書追加以外の実装変更が入った場合は, 設計と進捗を統合した `README.md`, `TypePM/AxiomAudit.lean` と照合して更新する.